Cairo University
Faculty of
Engineering
Computer Engineering Department
CMPS 102 Spring 2026

CMPS102
Programming Techniques
Project

Robo Rally

Introduction

In this project, we will build a game application called 'Robo Rally'. There are **two modes** in the game: **Design mode** is where the user is able to build the playing grid and **Play mode** is where he plays the game itself. In Robo Rally, two players move across a grid filled with various boosters and obstacles. The player's goal is to get to the cell with a flag before anyone else. He also takes advantage of the special cells to boost his game and obstruct opponents. The game ends when a player reaches the cell with a flag.

Your Task: You are required to write **C++ code** for this Game. Delivering a working project is NOT enough. You must use ***object-oriented programming*** to implement this application and respect the **responsibilities** of each class as specified in the document. See the evaluation criteria in phase 2 document for more information.

NOTE: The application should be designed so that the types of items and types of operations can be easily extended (*using inheritance*).

Project Schedule

Project Phase	***Deliverables***
Phase 1 [4 marks]	**Input-Output Classes**
Phase 2 [11 marks]	**Final Project Delivery**

NOTE: Number of students per team = **3 to 4 students (from same tutorial time).**

The project code must be totally yours. The penalty of cheating from any other source is not ONLY taking ZERO in the project grade but also taking up to MINUS seven and half grades (- 7.5) from the other class work grades.

Game Description

In the Design mode, the player can add, copy, cut, paste, and delete any game object. He can also save a grid or load a previously saved grid.

Players move within a grid of 55 cells, organized into 5 rows and 11 columns, with each cell numbered from 1 to 55.

Grid cells may contain game objects which are:

1. **Flag:** Destination for robots (players); reaching it wins the game
2. **Antenna:** Determines the turn of the players
3. Start or End of a **Belt:** Moves robots from one cell to another
4. **Danger zone**: Cause damage to robots
5. **Water Pits**: Robot reset to starting cell with health decreased with 3 points
6. **Workshop**: Allows robots to repair
7. **Rotating Gear**: Rotates the robot 90 degrees

Player:

- The Game has two players.
- Each player has:
  - Steps' count (initially step 1). This holds the value of the current cell the player is in.
  - Health (initially it is equal to 10).

- Players take turns, one after another to move their robot, the turn of the players is determined by the antenna (which is a game object placed in the grid, and we will discuss it later).
- At each turn a number of commands appear based on the health of the player
- Players move their robots by selecting movement commands, controlling how the robot navigates the map and interacts with the environment.
- Each robot executes its set of commands in order every round, giving a strategic element to movement and positioning.
- The player's robot moves according to the selected commands, and if the robot reaches a cell with a game object, the robot interacts with it.
- The player's target in the game is to reach the cell with the flag before any opponent reaches it.
- **Winning Player** --> is the first player to reach the cell with the flag

Movement Commands

As we mentioned before, the player moves his robot by selecting movement commands.

- The available commands to select from appear randomly and their count is based on the player's health. In this example the player's health is 10.
- A player has to select some commands to be executed by the robot .The number of commands a robot is allowed to execute during each round should only be **5 commands and as the health of the player decreases the number of commands it is allowed to execute decreases, it is calculated as Min(5, health)**

- The available commands that could appear to the player are:
  - Move Forward (X steps): Moves the robot X steps forward. (X is a random number between 1 and 3)
  - Move Backward (X steps): Moves the robot X steps backward. (X is a random number between 1 and 3)
  - Rotate Left: Rotates the robot 90 degrees to the left. (counter-clockwise)
  - Rotate Right: Rotates the robot 90 degrees to the right. (clockwise)

Game Objects

A Game cell contains players' objects and game objects. **It can hold multiple player objects, and at most one game object.**

A Game object is the fixed object placed in the grid's cells. **You cannot place game objects in the first or last cell.**

There are 7 types of Game Objects:

1. Flag:
    a. The flag marks the target cell that players aim to reach to win the game.
    b. Only one flag can exist on the grid, and design mode should include a validation to prevent multiple flags.

2. Water Pits:
    a. Cells having a water pit should be filled with dark blue color
    b. Water pits drown the robots so if a robot steps inside a water pit, it is reset to the starting cell with health points reduced with 3 points
3. Danger Zones:
    a. Cells having a danger zone should be filled with red color
    b. Danger zones reduce the health of the robots by 1 if a robot steps into them
4. Belt:
    a. Belts occupy exactly two cells, the start and end cells.
    b. When a player moves to the start cell, it will transfer him to the end cell.
    c. For simplicity, the start and end cell should be in the same column or the same row. (belt can be either vertical or horizontal)
    d. The start cell cannot be a special cell (a cell that contains other game object)
    e. The end cell can't contain a flag
    f. The end cell of a belt cannot be the start cell of another belt.
5. Workshop
    a. Robots can visit workshops on the map to repair themselves
6. Antenna
    a. The antenna determines the turn order for players.
        i. The distance of each player from the antenna determines his turn order in each round. The player with the minimum distance from the antenna will have the first turn in that round. If there is a tie, player number will break it, for example if player 1 & 2 are in the same cell and they have the same distance from the antenna, player 1 will play first in that round.
    b. Note that the turn order is calculated at the beginning of each round.
    c. Only one antenna can exist on the grid, and design mode should include a validation to prevent multiple antennas.
7. Rotating Gears
    a. Can be clockwise or anticlockwise
    b. Rotates the robot 90 degrees clockwise or anti-clockwise based on the type of the object

---

Main Toolbar Operations

The application supports 2 modes: **Design Mode** and **Play Mode**. Each mode contains 3 bars: a **toolbar** that contains the main operations of the current mode, a **status bar** for displaying messages to the user and a **commands bar** that shows the commands that the current player can choose from and the chosen commands (only shows it in the play mode, it's blank in the design mode). The application should support the following operations (actions) in each mode.

[Important Notes]:

- *Each operation in any of the following modes **must** have an **icon** (or more) in the toolbar. The user should click on the operation icon from the toolbar to choose it.*
- *For each action requiring a cell position, the user clicks on the cell **after** selecting the action icon in the toolbar.*
- **In all the following actions, any dynamically allocated memory MUST be deallocated.**

The main operations supported by the application are the following.

[I] Design Mode:

The purpose of this mode is to design the Grid of the game that you will play later. You specify in this mode the location of all game objects. The operations supported by this mode are:

1. Add Flag
    a. Places a single flag on the grid as the destination cell for players.
    b. Instructions:
    The status bar displays:
    `Click on the cell to add a flag` (to instruct the user to click on the cell to add the flag).
    c. Validation:
    Only one flag is allowed on the grid. If a flag already exists, adding another flag is not permitted.
2. Add Antenna
    a. Places an antenna on the grid, determining turn order during gameplay. (refer to the game objects section for more details)
    b. Instructions
    The status bar displays:
    `Click on the cell to place the antenna` (to instruct the user to click on the cell to place the antenna).
    c. Validation:
    If the cell already contains another game object, adding an antenna is not allowed.
3. Add Belt
    a. Adds a belt between two selected cells to move players from the start cell to the end cell.
    b. Instructions
    Status bar displays:
    `Click on the belt's start cell, then on the end cell` (to instruct the user to click on the belt's start cell, then on the end cell).
    c. Validations:
        i. The start cell can't be the first cell (cell #1)
        ii. The cells must be in the same row or column.

iii. The start cell cannot contain another game object, and the end cell cannot contain a flag.
iv. The end cell of one belt cannot be the start of another.

4. Add Water Pit
    a. Adds a water pit to the selected cell, which instantly drowns the robot and resets it to the starting cell with reduced health points
    b. **Instructions**:
        Status bar displays:
        `Click on the cell to add a water pit` (to instruct the user to click on the cell to add a water pit).
        The cell will change to blue if added successfully.
5. Add Danger Zone
    a. Adds a danger zone to the selected cell, reducing the health of any robot that steps into it.
    b. Instructions:
        Status bar displays:
        `Click on the cell to add a danger zone` (to instruct the user to click on the cell to add a danger zone).
        The cell will change to red if added successfully.
6. Add Workshop
    a. Places a workshop on the selected cell, allowing players to repair.
    b. Instructions:
        Status bar displays:
        `Click on the cell to add a workshop` (to instruct the user to click on the cell to add a workshop).
7. Add Rotating Gear (Clockwise or Counterclockwise)
    a. Adds a rotating gear to the selected cell, rotating robots that step on it in the chosen direction.
    b. Instructions:
        Click on the cell to add a rotating gear, then select the rotation direction (clockwise or counterclockwise).
8. Copy Game Object
    a. Copies a game object from a selected cell to the clipboard for future placement.
    b. Instructions: Click on the source cell to copy the game object.
    c. Validation: The source cell must contain a game object to be copied.
9. Cut Game Object
    a. Cuts a game object from a selected cell to the clipboard, removing it from the original cell.
    b. Instructions: Click on the source cell to cut the game object.
    c. Validation: The source cell must contain a game object to be cut.
10. Paste Game Object
    a. Pastes a copied or cut game object from the clipboard into a selected cell.
    b. Instructions: Click on the destination cell to paste the game object.
    c. Validation: The destination cell must not already contain another game object.
11. Delete Game Object
    a. Deletes the game object in the selected cell.
    b. Instructions: Click on the cell to delete its game object.
    c. Validation: The cell must contain a game object to be deleted.
12. Save Grid
    a. Saves the current grid, including all placed game objects, to a file.
    b. Instructions: Enter a filename to save the grid. If a file with that name exists, it will be overwritten.
13. Load/Open Grid
    a. Loads a saved grid file and displays it on the grid. If a grid is currently displayed, it will be cleared before loading the new one.

   b. After loading, the user can edit the loaded grid and continue the application normally.
   c. Instructions: Enter the filename of the grid to open.
   d. Validation: If the specified file cannot be found or loaded, an error message appears.
14. Switch to Play Mode
   a. Switches from Design Mode to Play Mode, enabling gameplay functions.
   b. Instructions: Select this option to switch modes.
15. Exit
   a. Exits the application, performing any necessary cleanup.
   b. Instructions: Select this option to close the application.

Note: **For all actions that involves adding/pasting game object handle the following cases**

1- if user selects the start cell:
   - The error message should appear “can’t insert an object in the start cell”
2- if the user selects a cell that contains another game object
   - The error message should appear “a game object already exists in this cell”

[II] Play Mode:

In Play Mode, two players compete by navigating their robots across the grid to reach the target flag cell. Both players start at the **start cell**, and their objective is to be the first to reach the cell containing the flag.

At each round, players take turns to take actions to help them reach their goal. The round consists of two phases; movement phase and shooting phase. The turn order is determined by the antenna, which was mentioned before. Movement phase begins where players have 4 options: select movement commands, execute the selected commands, reboot and repair, or use consumables they’ve purchased. During the movement phase, players will interact with game objects on the grid, such as belts, danger zones, and workshops. The game ends when a robot reaches the flag cell, and the first player to do so is declared the winner.

The current player's turn should be displayed in the **top right corner** during their turn, beside player's info. Additionally, available commands, and selected ones should be visible on the command bar.

Operations in Play Mode

At the beginning of each round, players have the following options:

1. **Select Movement Commands**
   - Players can choose from the available movement commands displayed each round.
   - Commands available to players are limited based on their health level. For example, if a player has 8 health points, they will have 8 commands to choose from.
   - Players may select commands to add to their saved list, allowing them to execute these movements later in the round.
   - Commands are randomly generated each round, and players must select from the available options. they can have multiple commands of the same type.
   - A player can select up to 5 commands to execute during the round. If a player has health points less than 5, he can only select the number of commands equal to his health points.
   - A player can select zero commands if he wish to skip the movement phase of the round. In this case, he shouldn't select this action and use the execute saved commands action instead.
2. **Execute Saved Commands**
   - After selecting movement commands, players can execute the saved commands to move their robot.

- The robot moves according to these saved movement commands in the order they were selected.
- The robot interacts with game objects on the grid based on the final position after executing each command. For example, if the player has 3 saved commands of moving forward, the robot will move 1 step forward to execute the first command, then if the new cell contains a game object, the robot will interact with it, then the robot will move 1 step forward, and interact with the new cell if it contains a game object, and so on.
- If the robot reaches a **workshop** cell, the player interacts with it only after all saved movement commands have been executed.

3. **Reboot & Repair**
   - The player can choose to reboot and repair their robot, restoring some health points.
   - If this option is selected, the robot will not move during this round.
4. **New Game**
   - Restarts the game, resetting players' positions, health, and any saved commands. The game begins again with **player 1**.
5. **Switch to Design Mode**
   - Players can switch from Play Mode to Design Mode at any time.
   - The game is ended, if the player switches to Design Mode, the game is reset and the grid is cleared.
   - The current grid is opened in Design Mode for editing, and any necessary cleanups are performed.
6. **Exit**
   - Exits the application and performs all necessary termination housekeeping tasks.

Additional Game Actions

Once movement actions are completed, the following game mechanics are triggered:

1. **Interactions with Game Objects**

   - If a robot reaches a cell containing a game object, the player can interact with it. Possible game objects include:
     - **Workshop**: Players robots' will be repaired
     - **Belts**: The robot is moved to another cell, based on the belt's direction.
     - **Danger Zone**: The robot loses health points if it lands here.
     - **Water Pit**: The robot is drowned leading to resetting player's position to starting cell with health reduced to 3 points.
     - **Rotating Gear**: Player's direction is rotated
2. **Winning Condition**
   - The game ends when a robot successfully reaches the flag cell. The first player to do so is declared the winner.

Main Classes of phase 1

You are given a ***code framework*** where we have ***partially*** written code of some of the project classes. For the graphical user interface (GUI), we have integrated an open-source ***graphics library*** that you will use to easily handle GUI (e.g. drawing grid on the screen and reading the coordinates of mouse clicks …etc.).

You should **stick to** the **given design** (i.e. hierarchy of classes and the specified job of each class) and complete the given framework as described in phase 1 deliverables section below.

.

Input Class:
ALL user inputs must come through this class. If any other class needs to read any input, it must call a member function of the input class. You should add suitable member functions for different types of inputs.

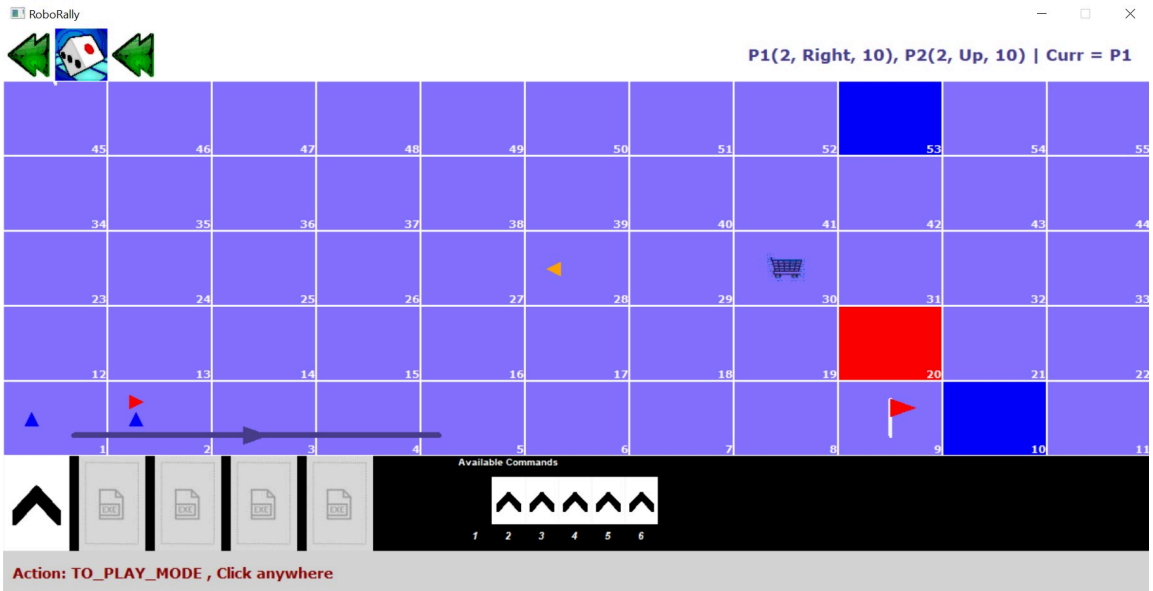
Output Class:
This class is responsible for **ALL** GUI outputs. It is responsible for toolbar and status bar creation, grid and game objects drawing, and for messages printing to the user. If any other class needs to make any output, it must call a member function of the output class. You should add suitable member functions for different types of outputs.

Notes: - No input or output is done through the console. All must be done through the **GUI window**.
- Input and Output classes are the **ONLY** classes that have **access to the GUI library**.

Cell position Class:

This class represents the cell position in the grid by having two data members called vCell and hCell. This class does NOT deal with real coordinates, it deals with the vCell, hCell and cellNum instead.

Program Interface



Example of the program interface

Project Phases

A partially-implemented code framework for Phase 1 and for Phase 2 are given to you to complete them.

For **fast navigation** in the given code in **Visual Studio**, you may need the following:

- ❑ **F12 (go to definition):** to go to definition of functions (code body), variables, …etc.
- ❑ **"Ctrl" then "Minus":** to return to the previous location of the cursor.
- ❑ **"Ctrl" + "Shift" + "F"**: Searches for a specific string in the entire project. (You can use it to search for "TODO" comments in the project).

1- Phase 1 (Input / Output Classes)

In this phase, you will implement the **Input** and the **Output** classes as they do not depend on any other classes. The **Input** and **Output** classes should be **finalized** and ready to run and test. Any expected user interaction (input/output) that will be needed by phase 2, should be implemented in phase 1.

Input and Output Classes Code and Test Code

You are given a code for phase 1 (separate from the code of the whole project) that contains both the input and output classes partially implemented. Each team should complete such classes as follows:

1- *Output Class*:

The **Output** class handles all GUI output tasks, including drawing the grid, toolbars, and game objects, and managing the display of player information.

Draw the Full Toolbars:

The **Output** class should create three full toolbars: one for **Design Mode** and one for **Play Mode**, and another one for the Commands **bar** which is available in play mode, each containing icons for every action available in that mode.

[Note]:
All project icons must be added in **Phase 1**.

Implement the Following Functions:

- ❑ **GetCellStartX(cellpos)** & **GetCellStartY(cellpos)**: Retrieve the X and Y coordinates of the upper-left corner of the specified cell position.
- ❑ **CreateDesignModeToolBar()** & **CreatePlayModeToolBar()**: Set up the toolbar icons for both Design and Play Modes, ensuring that all necessary icons are loaded.
- ❑ **DrawCell(cellPos, cellColor)**: Draw a cell rectangle with its cell number at the specified grid position.
- ❑ **DrawPlayer(cellPos, playerNum, playerColor, direction)**: Render a player triangle in the specified cell, with its position determined by `playerNum`.
- ❑ **DrawBelt(fromCellPos, toCellPos)**: Draw a directional belt from a starting cell to an ending cell.
- ❑ **DrawFlag(cellPos)**: Place a flag symbol in the designated cell.
- ❑ **DrawRotatingGear(cellPos, clockwise)**: Render a rotating gear image in the cell,

oriented based on the `clockwise` parameter.

- ❑ **DrawWorkshop(cellPos)**, **DrawAntenna(cellPos)**, **DrawDangerZone(cellPos)**, and **DrawWaterPit(cellPos)**: Draw the respective game objects or environmental features at the specified cell positions.
- ❑ **PrintPlayersInfo(info)**: Display player information on the right side of the toolbar.

Please refer to `Output.h` and `Output.cpp` for other functions that may be useful in completing these requirements.

2- *Input Class:*

The **Input** class captures all user inputs from the GUI, directing them to appropriate actions based on the user's interactions.

Implement the Following Functions:

- ❑ **GetUserAction()**: Detect user actions based on the clicked coordinates and map them to corresponding `ActionTypes`.
- ❑ **GetInteger()**: Capture an integer input from the user.
- ❑ **GetCellClicked()**: Capture a mouse click from the user and return the cell position clicked. If the click is outside the grid, return `(-1, -1)`.

Explore `Input.h` and `Input.cpp` for additional input functions and helper utilities.

3- *CellPosition Class:*

The **CellPosition** class defines the location of cells in the grid, with methods to convert between cell positions and cell numbers.

Implement the Following Functions:

- ❑ **SetVCell(v)** & **SetHCell(h)**: Setters for `vCell` and `hCell`, setting them only if within grid bounds.
- ❑ **IsValidCell()**: Check if the cell position is valid within the grid.
- ❑ **GetCellNumFromPosition(cellPosition)**: Calculate the cell number based on a cell position.
- ❑ **GetCellPositionFromNum(cellNum)**: Convert a cell number into its corresponding `CellPosition`.
- ❑ **AddCellNum(addedNum, direction)**: Add a specified number of cells to the current cell, adjusting for the specified direction.

Refer to the remaining code in `CellPosition.h` and `CellPosition.cpp` to understand additional functionality.

4- *DEFS.h and UI_Info.h Files*: as you implement the Input and Output class, you will find some parts you need to add in DEFS.h and UI_Info.h files.

5- *Test Code:* this is <u>not</u> part of the input or the output classes; it is just a test code (the **main**).

- ❑ Complete the code given in **TestCode.cpp** file to test both Input and Output classes.
- ❑ Do NOT re-write the main function from scratch, just complete the required parts.

Notes:

- In Phase 1 code, Follow any **///TODO** comments to know what should be done in each .h or .cpp file.
- Nearly all coordinate calculations are done for you. You just need to use it to draw every object correctly according to its specifications.

Notes on The Project Graphics Library*:*

- The origin of the library's window **(0, 0)** is at the **upper left** corner of the window.
- The direction of **increasing** the **x coordinate** is to the **right**.
- The direction of **increasing** the **y coordinate** is **down**.
- The only image extension that the library accepts is "**jpg**".
- You can use any alternative graphics library if you want, but with the condition of keeping the same interface (function prototypes) of the **Input** and **Output** classes and any other classes of phase 2 (same interface but the function body will be using your new library). However, we as TAs do not guarantee the support if you have any problem with your new library. It is your responsibility to deal with it.

Phase 1 Deliverables*:*

On the discussion day, each team should submit phase 1 code that has Input, Output, and CellPosition classes, DEFS.h, UI_Info.h and test program completed. If there is any further instructions it will be posted on the classroom.